PRD

- DropTune V1 — Product Requirements Document (PRD)
  - Overview
  - Problem Statement
  - Core Product Philosophy
    - Distributed Libraries
    - Ephemeral Sessions
    - Host-Based Playback
    - Pull-Based Media Transfer
    - Local-First
  - Definitions
    - Room
    - Host
    - Guest
    - Shared Library
  - Functional Requirements
    - 1. Room Creation
    - 2. Device Joining
    - 3. Library Indexing
    - 4. Global Search
    - 5. Queue Management
    - 6. Playback Hosting
    - 7. File Matching
    - 8. Media Transfer
    - 9. Playback State Synchronization
  - Non-Goals (V1)
  - Technical Constraints
    - Browser-First
    - User-Granted File Access
    - Temporary Sessions
    - Browser Limitations
  - Success Criteria
  - Future Expansion Possibilities
  - Architecture Plan (V1)
    - High-Level Architecture
    - Core Design Principle
    - 1. Room Coordinator
    - 2. Browser Clients
    - 3. Control Plane (WebSocket)

4. Data Plane (WebRTC)
5. Playback Engine
File Resolution Flow
Prefetch Strategy
State Model
Recommended V1 Tech Stack
Frontend
State Management
Styling
Realtime
P2P Transfer
Playback
Why NOT Electron/Tauri Initially
Biggest Known Risks

DropTune V1 — Product Requirements Document (PRD)

Overview

DropTune is a browser-first, local-network collaborative music playback system.

Multiple devices on the same network can:

- share local music libraries
- collaboratively build a queue
- choose a playback host device
- transfer audio files peer-to-peer on demand
- control playback state together

The system is designed around:

- local-first operation
- no mandatory installs
- browser accessibility
- temporary collaborative sessions
- pull-based media transfer

- MP3/FLAC-first workflows

Future support for streaming platforms (Spotify, YouTube Music, etc.) should be architecturally possible but is not part of V1.

Problem Statement

Users often have:

- fragmented local music libraries across devices
- Bluetooth speakers attached to one device
- songs available only on certain devices
- collaborative listening scenarios within homes, dorms, parties, or workspaces

Existing systems:

- centralize music ownership
- require uploads or syncing
- rely on streaming subscriptions
- require native installs
- do not handle distributed local-file libraries elegantly

DropTune aims to solve:

> “Play music collaboratively across devices using whatever files already exist locally on those devices.”

Core Product Philosophy

Distributed Libraries

Every device contributes metadata about its locally shared music.
No centralized library exists.

Ephemeral Sessions

Rooms are temporary collaborative playback sessions.
No accounts required.

Host-Based Playback

At any given time:

- one device acts as playback host
- playback physically occurs only on that device
- all devices can collaboratively control queue and playback state

---

Pull-Based Media Transfer

Files transfer only when required by the active host.
No full-library syncing.

Local-First

The system prioritizes:

- LAN operation
- peer-to-peer transfer
- browser accessibility
- minimal infrastructure

Definitions

Room

A collaborative playback session shared across devices.

Contains:

- participants
- queue
- playback state
- active host
- library metadata index

Host

The device currently responsible for:

- audio playback
- queue execution
- media fetching
- playback state broadcasting

Only one host may exist per room at a time.

Guest

Any connected device not currently acting as host.

Guests can:

- browse libraries
- search metadata
- add to queue
- control playback state

Shared Library

A voluntarily shared set of folders/files from a device.

The browser indexes metadata only.
Files remain local unless requested.

Functional Requirements

1. Room Creation

Users can:

- create a room from browser
- receive a local network URL and/or QR code
- allow other devices on LAN to join

No authentication required for V1.

---

2. Device Joining

Devices can:

- open room URL
- join room as participant
- optionally share local music folders

---

3. Library Indexing

Users can:

- choose folders/files manually

System indexes:

- filename
- track name
- duration
- format
- basic metadata if available

Supported formats:

- MP3
- FLAC
- WAV (optional)
- M4A/AAC (optional)

---

4. Global Search

All shared metadata across all devices becomes searchable room-wide.

Search results should indicate:

- source device(s)
- format
- duration

---

5. Queue Management

Participants can:

- add tracks to queue

- reorder queue
- remove tracks
- clear queue

Queue is shared room-wide.

---

6. Playback Hosting

Any device may become host.

Host responsibilities:

- execute playback
- request media
- maintain playback state
- preload upcoming queue items

Host switching should preserve:

- queue
- room state
- playback position where possible

---

7. File Matching

When host needs a track:

- check local library first
- compare by filename and/or track name

if local equivalent exists:

- play local version

otherwise:

- request transfer from source device

No advanced fingerprinting in V1.

---

8. Media Transfer

Transfers are:

- peer-to-peer
- pull-based
- temporary

Host requests media only when required.

Host may:

- preload next N tracks
- discard transferred files after session

Streaming vs full temporary transfer can be implementation-defined.

---

9. Playback State Synchronization

All participants can:

- play/pause
- seek
- skip

Host remains authoritative playback executor.

Shared state includes:

- current track
- elapsed time
- playback status
- queue state

Non-Goals (V1)

Not included in V1:

- cloud sync
- internet rooms
- accounts
- permanent library sync
- Spotify integration

- YouTube integration
- playlist persistence
- recommendation systems
- advanced identity matching
- background daemon services
- automatic LAN discovery
- iOS-specific optimization

Technical Constraints

Browser-First

V1 must function entirely within browser.
No native app requirement.

User-Granted File Access

All file access must be explicitly granted by user.
No automatic filesystem scanning.

Temporary Sessions

Room state may be memory-only in V1.
Persistence optional.

Browser Limitations

Known constraints:

- mobile browser sleeping
- autoplay restrictions
- limited background execution
- inconsistent filesystem APIs
- browser storage limitations

These are accepted tradeoffs for V1.

Success Criteria

V1 is successful if users can:

1. Create a room
2. Join from multiple devices
3. Share local folders
4. Search all shared tracks
5. Queue tracks collaboratively
6. Switch playback host
7. Play transferred/local tracks successfully
8. Maintain reasonably responsive playback controls on LAN

Future Expansion Possibilities

Potential future iterations:

- Spotify adapters
- YouTube Music adapters
- internet rooms
- persistent identities
- native apps
- automatic LAN discovery
- advanced matching/fingerprinting
- offline caching
- synchronized playback across multiple speakers
- hybrid browser/native nodes

Architecture Plan (V1)

High-Level Architecture

DropTune V1 consists of:

1. Room Coordinator
2. Browser Clients
3. WebSocket Control Plane
4. WebRTC Transfer Plane
5. Local Playback Engine

Core Design Principle

Separate:

- Control Plane
- from
- Data Plane

This is extremely important.

---

1. Room Coordinator

One browser tab temporarily acts as coordinator.

Responsibilities:

- room state
- participant registry
- queue authority
- playback state propagation
- WebRTC signaling

Coordinator is NOT necessarily playback host.
Coordinator can migrate later if needed.

---

2. Browser Clients

Each device/browser:

- joins room
- optionally shares folders
- exposes metadata

- participates in transfer network
- can become playback host

---

3. Control Plane (WebSocket)

Used for:

- room state updates
- queue events
- playback events
- participant presence
- signaling for WebRTC

Messages include:

- JOIN_ROOM
- QUEUE_ADD
- PLAY
- PAUSE
- SEEK
- HOST_CHANGE
- FILE_REQUEST
- FILE_AVAILABLE

Low bandwidth.
Realtime-oriented.

---

4. Data Plane (WebRTC)

Used ONLY for:

- media transfer
- chunk streaming
- prefetching

Peer-to-peer.
Avoids routing large files through coordinator.

---

5. Playback Engine

Host device:

- maintains playback element
- manages preload buffer
- resolves local-vs-transfer playback
- broadcasts playback state

Playback engine remains abstracted from transfer layer.

File Resolution Flow

When queue advances:

1. Host checks local indexed library
2. Match via filename/track name
3. If match exists:
    - play local file
4. Else:
    - request media from source device
5. Source streams/transfers via WebRTC
6. Host buffers and plays

Prefetch Strategy

Host maintains:

- current track
- next N queued tracks

Background preload:

- reduces playback gaps
- tolerates WLAN instability
- reduces latency

State Model

Queue entries should be abstract media references.

Example:

```
{
  "track_name": "Numb",
  "sources": [
    {
      "device_id": "phone",
      "path_hint": "...",
      "format": "mp3"
    }
  ]
}
```

Queue must survive:

- host changes
- disconnects
- partial availability

Recommended V1 Tech Stack

Frontend

- React
- TypeScript
- Vite

Reason:

- fast iteration
- mature ecosystem
- strong WebRTC/WebSocket support

State Management

Zustand

Simple and lightweight.

Styling

Tailwind CSS

Realtime

WebSocket server using Node.js

Could initially run:

- alongside coordinator
- or lightweight standalone LAN process

P2P Transfer

WebRTC Data Channels

Playback

HTMLAudioElement initially

Simple and browser-native.

Why NOT Electron/Tauri Initially

Native apps add complexity around:

- packaging
- permissions
- platform differences
- updates
- mobile parity

Browser-first is better for discovering:

- session UX
- queue semantics
- transfer behavior
- host switching

before committing to native architecture.

Biggest Known Risks

1. Mobile browser sleeping
2. Browser file API inconsistencies
3. Large FLAC transfer reliability
4. WebRTC edge cases on LAN
5. Playback interruption during host migration
6. Background tab throttling

These are acceptable V1 tradeoffs.